MONGODB

Practical Hands-on Workbook

Beginner to Intermediate | Mongosh + MongoDB Compass

Purpose	Practice database creation, collections, documents, CRUD, queries, aggregation and indexing.
Scenario	An E-commerce database containing customers, products and orders.
Suggested duration	3–4 hours
Tools	MongoDB Community Server, mongosh and MongoDB Compass

Learning objectives

• Create databases and collections.

• Insert, retrieve, update and delete documents.

• Use filters, comparison operators, projection, sorting and limits.

• Build aggregation pipelines and join collections using $lookup.

• Create and inspect indexes and interpret query execution statistics.

Before you start: Ensure MongoDB Community Server is running and mongosh is installed. Open a terminal and launch `mongosh`.

1. Create a Database and Collections

Step 1 — Switch to or create the database

```
use ecommerce
```

MongoDB creates the database when data is first stored in it.

Step 2 — Create collections

```
db.createCollection("customers")
db.createCollection("products")
db.createCollection("orders")
```

Step 3 — Verify collections

```
show collections
```

2. Insert Documents

Step 4 — Insert customer records

```
db.customers.insertMany([
  { customer_id: 101, name: "Rahul Sharma", city: "Delhi", age: 28, email: "rahul@gmail.com" },
  { customer_id: 102, name: "Priya Verma", city: "Mumbai", age: 32, email: "priya@gmail.com" },
  { customer_id: 103, name: "Amit Kumar", city: "Delhi", age: 25, email: "amit@gmail.com" },
  { customer_id: 104, name: "Sneha Rao", city: "Bangalore", age: 29, email: "sneha@gmail.com" },
  { customer_id: 105, name: "Vikas Singh", city: "Jaipur", age: 35, email: "vikas@gmail.com" }
])
```

Step 5 — Insert product records

```
db.products.insertMany([
  { product_id: 201, name: "Laptop", category: "Electronics", price: 55000, stock: 10 },
  { product_id: 202, name: "Mobile", category: "Electronics", price: 25000, stock: 20 },
  { product_id: 203, name: "Chair", category: "Furniture", price: 5000, stock: 15 },
  { product_id: 204, name: "Table", category: "Furniture", price: 8000, stock: 8 },
  { product_id: 205, name: "Headphones", category: "Electronics", price: 2000, stock: 30 }
])
```

Tip: `insertMany()` inserts multiple documents in one operation. MongoDB automatically creates an `_id` field if one is not supplied.

3. Read Operations — Find Documents

Step 6 — Display all customers

```
db.customers.find()
```

Step 7 — Display documents in readable format

```
db.customers.find().pretty()
```

Step 8 — Find customers from Delhi

```
db.customers.find({ city: "Delhi" })
```

Step 9 — Find customers older than 30

```
db.customers.find({ age: { $gt: 30 } })
```

Step 10 — Find customers aged between 25 and 32

```
db.customers.find({ age: { $gte: 25, $lte: 32 } })
```

Step 11 — Display only customer name and city

```
db.customers.find({}, { _id: 0, name: 1, city: 1 })
```

4. Update Operations

Step 12 — Change customer 101's city

```
db.customers.updateOne(
  { customer_id: 101 },
  { $set: { city: "Chennai" } }
)
```

Step 13 — Increase customer 102's age by one

```
db.customers.updateOne(
  { customer_id: 102 },
  { $inc: { age: 1 } }
)
```

Step 14 — Add a membership field to all customers

```
db.customers.updateMany(
  {},
  { $set: { membership: "Regular" } }
)
```

5. Delete Operations

Step 15 — Delete one customer

```
db.customers.deleteOne({ customer_id: 105 })
```

Step 16 — Delete customers from Mumbai

```
db.customers.deleteMany({ city: "Mumbai" })
```

Warning: Delete operations permanently remove matching documents. Always run a matching `find()` query first to verify the filter.

6. Sorting and Limiting

Step 17 — Sort customers by age, ascending

```
db.customers.find().sort({ age: 1 })
```

Step 18 — Sort products by price, descending

```
db.products.find().sort({ price: -1 })
```

Step 19 — Display the first three products

```
db.products.find().limit(3)
```

7. Aggregation

Step 20 — Count customers city-wise

```
db.customers.aggregate([
  { $group: {
      _id: "$city",
      totalCustomers: { $sum: 1 }
  }}
])
```

Step 21 — Calculate average product price by category

```
db.products.aggregate([
  { $group: {
      _id: "$category",
      averagePrice: { $avg: "$price" }
  }}
])
```

Aggregation pipelines process documents through stages. Here, `$group` groups records and calculates summary values.

8. Indexing Operations

Step 22 — Create an ascending index on city

```
db.customers.createIndex({ city: 1 })
```

Step 23 — View all indexes

```
db.customers.getIndexes()
```

Step 24 — Create a compound index on city and age

```
db.customers.createIndex({ city: 1, age: -1 })
```

Step 25 — Create a text index on customer name

```
db.customers.createIndex({ name: "text" })
```

Step 26 — Search using the text index

```
db.customers.find({ $text: { $search: "Rahul" } })
```

Step 27 — Analyse query execution

```
db.customers.find({ city: "Delhi" })
  .explain("executionStats")
```

Step 28 — Drop the single-field city index

```
db.customers.dropIndex({ city: 1 })
```

Note: The compound index on `{ city: 1, age: -1 }` is different from the single-field city index. Dropping the single-field index does not remove the compound index. The text index remains in place.

9. Working with the Orders Collection

Step 29 — Insert order records

```
db.orders.insertMany([
  { order_id: 301, customer_id: 101, product: "Laptop", quantity: 1, amount: 55000, status: "Delivered" },
  { order_id: 302, customer_id: 102, product: "Mobile", quantity: 2, amount: 50000, status: "Pending" },
```

```
  { order_id: 303, customer_id: 103, product: "Chair", quantity: 2, amount: 10000, status: "Delivered" },
  { order_id: 304, customer_id: 104, product: "Headphones", quantity: 1, amount: 2000, status: "Cancelled" }
])
```

Step 30 — Find delivered orders

```
db.orders.find({ status: "Delivered" })
```

Step 31 — Calculate total sales across all orders

```
db.orders.aggregate([
  { $group: {
      _id: null,
      totalSales: { $sum: "$amount" }
  }}
])
```

Step 32 — Calculate sales by order status

```
db.orders.aggregate([
  { $group: {
      _id: "$status",
      totalAmount: { $sum: "$amount" },
      totalOrders: { $sum: 1 }
  }}
])
```

Step 33 — Join orders with customer details using $lookup

```
db.orders.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "customer_id",
      foreignField: "customer_id",
      as: "customerDetails"
    }
  }
])
```

The `$lookup` stage performs a left outer join between collections. If a customer was deleted earlier in this exercise, that order will have an empty `customerDetails` array.

10. Independent Practice Questions

1. Display all products belonging to the Electronics category.

2. Find products costing more than ■10,000.

3. Find customers whose age is less than 30.

4. Display the three most expensive products.

5. Increase the price of all Electronics products by 10%.

6. Find customers from Delhi or Chennai using $in.

7. Count the total number of products.

8. Calculate the maximum and minimum product price.

9. Find orders with an amount greater than ■20,000.

10. Create an index on product_id.

11. Compare query execution before and after indexing.

12. Find the total number of orders for each customer.

11. MongoDB Compass Practical

Repeat the operations in MongoDB Compass using the graphical interface. Select the ecommerce database and open the relevant collection.

Operation	MongoDB Compass location
Create database	Create Database
Create collection	Create Collection
Insert document	Add Data → Insert Document
Find records	Documents → Filter
Update records	Edit Document
Delete records	Select document → Delete
Create/view indexes	Indexes tab → Create Index
Aggregation	Aggregations tab
Explain query	Explain Plan

12. Expected Learning Outcomes

• Create databases and collections.

• Insert, retrieve, update and delete documents.

• Apply filtering and comparison operators.

• Perform sorting, projection and limiting.

• Build aggregation pipelines.

• Join collections using $lookup.

• Create single-field, compound and text indexes.

• Interpret COLLSCAN, IXSCAN and execution statistics.

13. Practical Assessment

Build a small E-commerce database using customers, products and orders. Complete the following tasks:

• Insert at least five customers, five products and four orders.

• Perform at least two insert, read, update and delete operations.

• Generate a sales summary using aggregation.

• Join orders with customers using $lookup.

• Create suitable indexes and compare query execution statistics.

• Demonstrate the same core operations in MongoDB Compass.

Trainer's tip: Use the Explain Plan to compare COLLSCAN and IXSCAN. Ask learners to compare `nReturned`, `totalKeysExamined` and `totalDocsExamined`. An index is not automatically faster for every query; selectivity and query shape matter.